Project 2: Personalized Search

Personalized Book Search Engine

Jonathan Tadesse
jtadesse@kth.se

April 2026

Abstract

We present a personalized book search engine built on Elasticsearch for the CMU Book Summary Dataset, which contains 16,559 books with plot summaries and metadata such as title, author, genre and publication year. The system combines a lexical BM25 baseline with a personalized reranking stage that uses two sources of user evidence: explicit profile information, consisting of favorite genres, authors, books and free-text interests, and implicit feedback derived from clicked results. In addition, dense semantic representations produced with the `all-MiniLM-L6-v2` embedding model are used to measure similarity between queries, user profiles and book summaries. At query time, the system first retrieves a candidate set using lexical matching and then reranks the results with a weighted scoring function that combines lexical relevance, preference matching, click-based evidence and semantic similarity. We evaluate the system using 12 synthetic personas, each with an explicit profile, three warm-up queries and two test queries, under three settings: baseline retrieval, personalized retrieval without clicks and personalized retrieval after clicks. Results measured with Precision@10 and nDCG@10 show that personalization is generally beneficial, with the largest improvements typically appearing after implicit feedback has been incorporated. The implementation is available at: GitHub.

1 Introduction

Information retrieval has long focused on ranking documents by their estimated relevance to a query. In classical retrieval systems, the ranking is based mainly on lexical evidence and BM25 became one of the most established ranking functions because of its strong empirical performance and clear relevance based interpretation [3].

As search systems evolved, researchers observed that the same query can reflect different intents for different users. This led to personalized search, where ranking is adapted using user-specific evidence rather than relying only on query-document matching. Earlier work showed that prior interactions and implicit behavioral signals can be used to re-rank search results, while later large-scale studies found that personalization can improve effectiveness for some queries but not uniformly for all users and search contexts [4, 1]. In this project, we study personalized search in the domain of books using the CMU Book Summary Dataset which contains 16,559 books with plot summaries and metadata : title, author and genre. Our system is implemented on top of Elasticsearch and uses BM25 for first stage retrieval, followed by a personalized re-ranking stage based on explicit user preferences and implicit click feedback.

The goal is to investigate whether user modeling can improve ranking quality over a non personalized lexical baseline. We represent explicit preferences through favorite genres, authors, books and free text interests, and we represent implicit preferences through clicked results. Although past queries are stored, they are not directly treated as preference signals, since a query may be exploratory or unsuccessful and therefore may not reflect what the user actually prefers. Clicked documents provide a more reliable behavioral signal because they indicate that the user showed at least some interest in the retrieved material.

2 Related Work

Personalized search has been studied as an extension of standard retrieval, where ranking is adapted using information about the individual user rather than relying only on the current query. Early work by Teevan et al. showed that user interests and prior activities can be used to personalize search results and better reflect individual information needs [4]. This line of work established the idea that the same query may require different rankings for different users. Later, Dou et al. proposed a large-scale evaluation of personalized search strategies using query logs and demonstrated that personalization can improve retrieval effectiveness, although the gains are not consistent across all queries and users [1]. Their findings are important because they show that personalization should not be assumed to help uniformly, but instead depends on the quality of the user model and the search context. A key distinction in personalized retrieval is between explicit and implicit user information. Explicit signals include directly stated preferences such as favorite topics, authors or items. Implicit signals are derived from user behavior, most commonly clicks. Joachims et al. showed that clickthrough data can be interpreted as implicit

feedback, while also emphasizing that clicks are noisy and affected by presentation bias [2]. For this reason, click data is useful as a behavioral signal, but it should not be treated as a perfect substitute for relevance judgments. More recent work has explored interpretable and lightweight user profiles for personalized retrieval. Torbati et al. showed that sparse and scrutable profiles can still be effective for personalized entity search [5]. This is relevant to our project, since our user profiles are also intentionally simple and interpretable, consisting of favorite genres, favorite authors, favorite books and click-derived preferences. Our approach follows the general re-ranking perspective in personalized search, but applies it in a book retrieval setting where preferences are often naturally expressed through authors, genres and previously liked books.

3 Method

3.1 System Overview

Our system is a personalized search engine built on Elasticsearch. It consists of four main components: document indexing, baseline retrieval, user profile construction and personalized reranking. At indexing time, each record in the CMU Book Summary Dataset is parsed into structured metadata fields and a dense semantic representation. At query time, Elasticsearch retrieves an initial candidate set using lexical matching, after which the system optionally reranks the candidates using user-specific evidence. The web interface is implemented in Flask and allows users to enter explicit preferences, submit queries and click results. Explicit preferences include favorite genres, favorite authors, favorite books and free-text interests. Each click is recorded in both a log index and a profile index, which allows the user profile to be updated continuously. Figure 1 illustrates the overall pipeline.

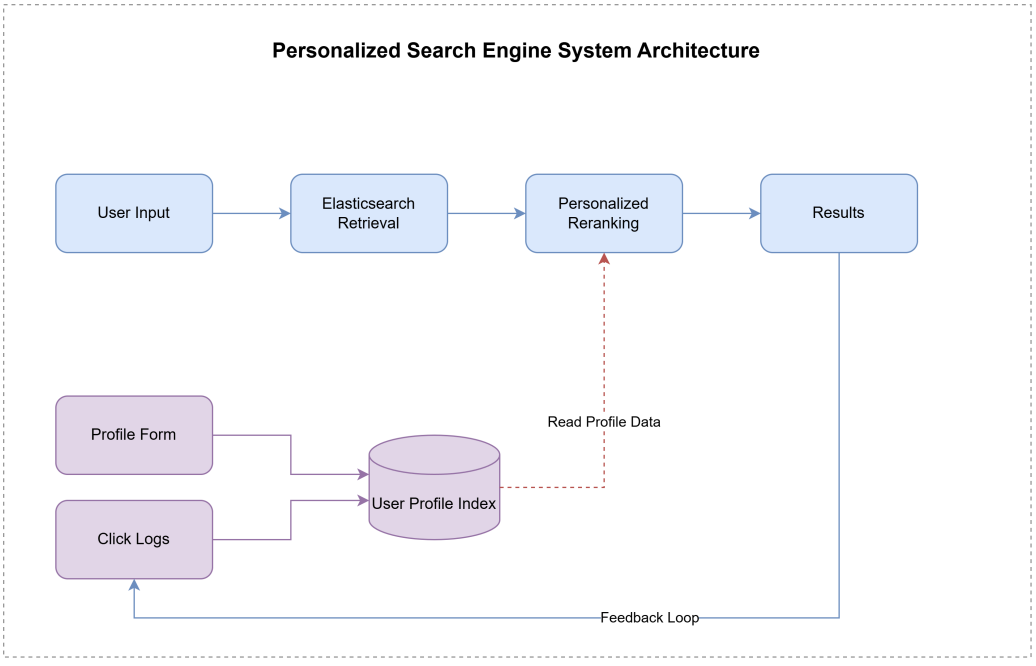

Figure 1: High-level architecture of the personalized search engine.

3.2 Document Collection

We use the CMU Book Summary Dataset, which contains 16,559 books with metadata and plot summaries. Each record includes a Wikipedia ID, Freebase ID, title, author, publication date, genres and a plot summary. The Wikipedia ID is used as the Elasticsearch document identifier. For semantic matching, we encode each document using `all-MiniLM-L6-v2`, which produces a normalized 384-dimensional dense vector. In our implementation, the embedding input is formed from the first 500 characters of the summary, prefixed with `"Summary: "`. The limit of 500 was chosen to shorten the amount of time for indexing while still producing useful information. The full metadata and the resulting dense vector are stored in the document index.

3.3 Elasticsearch Indices

The system maintains three Elasticsearch indices.

booksummaries. This index stores the book collection. The `title` and `author` fields are indexed as text fields with `keyword` subfields for exact matching. The `summary` field uses a custom English analyzer with lowercasing, possessive stemming, stopword removal and English stemming. The `genres` field supports both full-text and keyword-style matching. Each document also stores a 384-dimensional `dense_vector` field, `doc_vector`, with cosine similarity.

user_logs. This is an append-only log of click events. Each entry stores the user ID, query, clicked document ID, title, author, genres and timestamp.

user_profiles. This index stores one profile per user. It contains explicit preference fields such as `favorite_genres`, `favorite_authors`, `favorite_books` and `interests_text`, together with a concatenated profile text and its embedding vector. It also stores implicit feedback in the form of clicked document counts, click-based genre counts and click-based author counts. In addition, it stores recent queries, the total number of clicks and timestamps.

3.4 Baseline Retrieval

The baseline retrieval stage uses Elasticsearch `multi_match` in `best_fields` mode over `title`, `author`, `summary` and `genres`, with weights 3, 2, 1 and 1 respectively. Let $s_{\mathrm{BM25}}(q, d)$ denote the lexical relevance score of document d for query q. BM25 is given by

$$s_{\mathrm{BM25}}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t,d)(k_1+1)}{f(t,d) + k_1 \left(1 - b + b \frac{|d|}{\mathrm{avgdl}}\right)},$$

where $f(t, d)$ is the term frequency of term t in document d, $|d|$ is the document length and $avgdl$ is the average document length in the collection. When personalization is disabled, the

system returns the top k documents ranked by BM25. When personalization is enabled, it retrieves the top $K = 20$ candidates and passes them to the reranking stage.

3.5 User Profile Representation

Each user profile has an explicit and an implicit component.

Explicit profile. Users provide favorite genres, favorite authors, favorite books and free-text interests through the interface. These fields are merged into a profile text and encoded into a dense vector:

$$T_{\text{exp}} = \text{concat}(\text{genres}, \text{authors}, \text{books}, \text{interests}), \qquad \mathbf{u}_{\text{exp}} = E(T_{\text{exp}}),$$

where $E(\cdot)$ denotes the sentence embedding model. The profile can be saved either by replacing the previous profile or by merging new values with the old ones after normalization and deduplication.

Implicit profile. Each click updates four signals: clicked document counts, click-based genre counts, click-based author counts and a recent-query list. We decided to omit past queries as a ranking feature. A query may be exploratory, ambiguous or unsuccessful and therefore is not necessarily a reliable indicator of preference. For instance, a user might search for "Horror movies in forest settings" and not get relevant documents, but this does not mean that the retrieved results reflect the user's actual interests. Clicked results on the other hand provide a stronger behavioral signal because they indicate that the user showed at least some interest in the retrieved material. To derive an implicit semantic profile, the system selects the top 30 most-clicked documents and computes a click-weighted average of their document vectors:

$$\mathbf{u}_{\text{click}} = \frac{\sum_{i=1}^{n} w_i \mathbf{d}_i}{\left\| \sum_{i=1}^{n} w_i \mathbf{d}_i \right\|},$$

where w_i is the click count of document i and $\mathbf{d}_i$ is its stored embedding vector.

3.6 Adaptive Profile Blending

If both explicit and click-based vectors are available, they are combined using the total number of clicks n:

$$\alpha_{\text{exp}} = \frac{5}{n+5}, \qquad \alpha_{\text{click}} = \frac{n}{n+5},$$

$$\mathbf{u} = \text{normalize}\left(\alpha_{\text{exp}} \mathbf{u}_{\text{exp}} + \alpha_{\text{click}} \mathbf{u}_{\text{click}}\right).$$

This design gives full weight to the explicit profile at cold start and gradually increases the influence of implicit feedback as clicks accumulate. If only one vector is available, that vector is used directly. If neither vector exists, no user-level semantic score is added.

3.7 Personalized Reranking

For each candidate document d, the system computes a final personalized score as

$$S(q,d,u) = \tilde{s}_{\text{BM25}}(q,d) + s_{\text{genre}}(d,u) + s_{\text{author}}(d,u) + s_{\text{book}}(d,u) + s_{\text{repeat}}(d,u) + s_{\text{qsem}}(q,d) + s_{\text{usem}}(u,d),$$

where $\tilde{s}_{\text{BM25}}$ is the normalized lexical score and the remaining terms capture explicit preferences, click-based evidence and semantic similarity. The lexical score is normalized within the candidate set:

$$\tilde{s}_{\text{BM25}}(q,d) = 10.0 \cdot \frac{s_{\text{BM25}}(q,d)}{\max_{d' \in D_K(q)} s_{\text{BM25}}(q,d')}.$$

Genre matching. The genre score combines explicit favorite genres and click-derived genre preferences:

$$s_{\text{genre}}(d,u) = 2.2\,\alpha_{\text{exp}}\, m_g^{\text{exp}}(d,u) + 2.8\,\alpha_{\text{click}}\, m_g^{\text{click}}(d,u),$$

where $m_g^{\text{exp}}(d,u)$ is the number of matches between the document genres and the user's favorite genres, and $m_g^{\text{click}}(d,u)$ is the sum of normalized click-based genre preferences for the genres of d.

Author matching. The author score is defined as

$$s_{\text{author}}(d,u) = 1.8\,\alpha_{\text{exp}}\, m_a^{\text{exp}}(d,u) + 2.2\,\alpha_{\text{click}}\, m_a^{\text{click}}(d,u),$$

where $m_a^{\text{exp}}(d,u)$ is a binary indicator for whether the document author appears in the user's favorite authors, and $m_a^{\text{click}}(d,u)$ is the normalized click-based preference for that author.

Favorite-book matching. The favorite-book score uses explicit preferences only:

$$s_{\text{book}}(d,u) = 2.0\,\alpha_{\text{exp}}\, m_b(d,u),$$

where $m_b(d,u) \in [0,1]$ is a fuzzy title-overlap score. Exact title match gives score 1.0, substring containment gives 0.8, and partial token overlap is measured using the Dice coefficient.

Repeat-click bonus. Previously clicked documents receive an additional bonus with logarithmic damping:

$$s_{\text{repeat}}(d,u) = 1.6\,\alpha_{\text{click}} \log\left(1 + c(d,u)\right),$$

where $c(d,u)$ is the number of times user u has clicked document d.

Semantic similarity. The query is embedded as $\mathbf{q} = E(q)$. The query-document and user-document semantic scores are then computed using clamped cosine similarity:

$$s_{\text{qsem}}(q, d) = 4.0 \cdot \max\left(0, \cos(\mathbf{q}, \mathbf{d})\right),$$

$$s_{\text{usem}}(u, d) = 8.0 \cdot \max\left(0, \cos(\mathbf{u}, \mathbf{d})\right),$$

where $\mathbf{d}$ is the document embedding and $\mathbf{u}$ is the final user vector. The larger weight on s_{usem} reflects the design choice to prioritize alignment with long-term user interests during reranking. Candidates are finally sorted in descending order of $S(q, d, u)$.

3.8 Implementation Details

The system is implemented in Python using Flask, the Elasticsearch Python client and `sentence-transformers`. The main components are the web application, the indexing pipeline, the profile manager, the click logger and the reranking module. Document indexing uses the Elasticsearch bulk API with chunk size 200, while embedding generation is performed in batches of 32 documents. String comparisons for authors, genres and titles are normalized to avoid duplicate variants caused by differences in spacing or capitalization. Table 1 summarizes the reranking parameters used in the final system.

Table 1: Reranking weight parameters.

Parameter	Description	Value
W_{BM25}	Normalized BM25 weight	10.0
W_{eg}	Explicit genre weight	2.2
W_{cg}	Click genre weight	2.8
W_{ea}	Explicit author weight	1.8
W_{ca}	Click author weight	2.2
W_{book}	Favorite-book weight	2.0
W_{rep}	Repeat-click weight	1.6
W_{qsem}	Query-document semantic weight	4.0
W_{usem}	User-document semantic weight	8.0

The reranking weights are chosen to keep the score interpretable, so every ranking shifts can be traced to a user preference, click signal or sematic similarity.

4 Experimental Setup

4.1 Evaluation Design

We constructed 12 synthetic personas for the evaluation. Each persona contained four explicit preference fields: favorite genres, favorite authors, favorite books and a free-text interest description. In addition, each persona was assigned three warm-up queries relevant to the user profile. These warm-up queries were intended to generate interaction data for further personalization. Finally, each persona had two test queries used for evaluation.

Each test query was evaluated under three settings:

1. **Baseline**, where no user-specific information was provided to the search engine.
2. **No clicks**, where the search engine used only the persona's explicit profile information and no click history.
3. **After clicks**, where the search engine used both the explicit profile and the click-based information generated from the three warm-up queries.

For the baseline setting, we issued the two test queries for each persona and recorded the top 10 retrieved books. Next, we exposed the persona's explicit information to the search engine without clicking any books and issued the same test queries again. This produced the *No clicks* setting, which corresponds to the cold-start personalization scenario. We then used the three warm-up queries to simulate user interaction by clicking books that were relevant to the persona profile. Finally, we re-ran the same test queries with this click-based information included, producing the *After clicks* setting. Relevance assessment followed a pooled judgment procedure. For each persona and test query, we pooled the books retrieved across the three settings and manually assigned relevance labels to the unique documents in the pool. If a book appeared in more than one ranked list for the same query, the same relevance label was reused in all settings. This ensured consistent labeling across systems while reducing redundant judgments. Each persona file therefore contains

$$2 \text{ queries} \times 3 \text{ systems} \times 10 \text{ results} = 60 \text{ rows}.$$

The ranked lists were then compared using Precision@10 and nDCG@10 in order to measure the effect of personalization on retrieval quality.

4.2 Evaluation Metrics

We use Precision@10 and nDCG@10.

Precision@10. A document is relevant if its label is at least 1:

$$P@10 = \frac{1}{10} \sum_{i=1}^{10} \text{rel}_i$$

nDCG@10.

$$DCG@10 = \sum_{i=1}^{10} \frac{2^{r_i} - 1}{\log_2(i+1)}$$

$$nDCG@10 = \frac{DCG@10}{IDCG@10}$$

5 Results

5.1 Overall Performance

Table 2 reports the per-persona averages of P@10 and nDCG@10 under the three evaluation settings. On average, both metrics improve monotonically as more user evidence is incorporated. The transition from Baseline to No clicks, which corresponds to adding only explicit preferences, already accounts for roughly half of the total improvement in both metrics.

Table 2: Average per-persona performance under Baseline, No clicks, and After clicks.

	P@10				**nDCG@10**			
Persona	**Base.**	**No cl.**	**After cl.**	**Δ**	**Base.**	**No cl.**	**After cl.**	**Δ**
Persona 1	0.60	0.60	0.60	0.00	0.516	0.564	0.490	-0.026
Persona 2	0.65	0.75	0.85	+0.20	0.706	0.803	0.888	+0.181
Persona 3	0.70	0.75	0.75	+0.05	0.763	0.813	0.831	+0.068
Persona 4	0.60	0.60	0.65	+0.05	0.589	0.647	0.703	+0.115
Persona 5	0.50	0.60	0.60	+0.10	0.606	0.589	0.596	-0.010
Persona 6	0.60	0.70	0.80	+0.20	0.753	0.937	0.916	+0.162
Persona 7	0.55	0.60	0.60	+0.05	0.633	0.681	0.712	+0.079
Persona 8	0.85	0.80	0.90	+0.05	0.649	0.899	0.844	+0.194
Persona 9	0.60	0.70	0.75	+0.15	0.583	0.651	0.742	+0.160
Persona 10	0.55	0.60	0.65	+0.10	0.517	0.694	1.000	+0.483
Persona 11	0.55	0.75	0.80	+0.25	0.574	0.616	0.713	+0.140
Persona 12	0.70	0.80	0.85	+0.15	0.672	0.604	0.655	-0.016
Avg.	**0.621**	**0.688**	**0.733**	**+0.112**	**0.630**	**0.708**	**0.757**	**+0.127**

5.2 Per-Persona Analysis

Figure 2 shows the average values of P@10 and nDCG@10 across the two test queries for each persona. The majority of personas (9 out of 12) show a P@10 improvement from Baseline to After clicks, with Persona 11 (+0.25) and Personas 2 and 6 (+0.20 each) exhibiting the largest gains. However, Persona 1 is the only persona with a negative nDCG@10 delta (−0.026).

Two other personas (5 and 12) also show small negative nDCG@10 deltas, indicating that personalization retrieved more relevant documents but placed them in a less favorable order. Per-query breakdowns are provided in Appendix Table 4

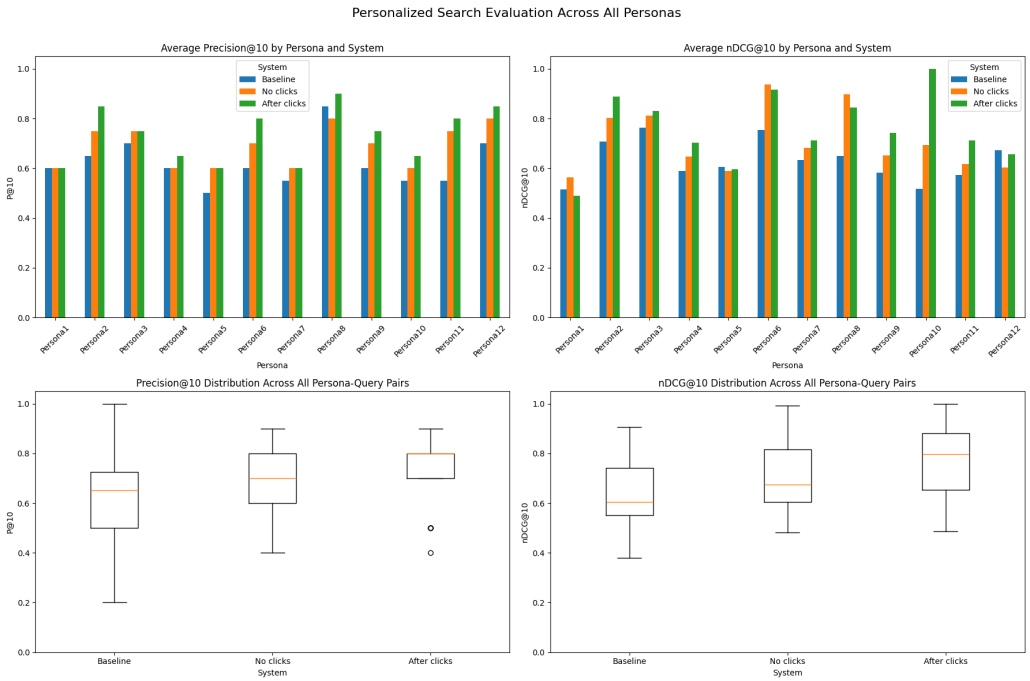

Figure 2: Performance across personas under Baseline, No clicks, and After clicks.

6 Discussion

Overall effectiveness. The results show a consistent improvement across both metrics as more user evidence is incorporated: P@10 rises from 0.62 (Baseline) to 0.69 (No clicks) to 0.73 (After clicks), and nDCG@10 from 0.63 to 0.71 to 0.76. An important finding is that the explicit profile alone already produces a meaningful gain without requiring any interaction history, which means the system is effective from the first query in a cold-start scenario. And several personas achieve large individual improvements: Persona 6 reaches 0.94 with explicit preferences alone, and Persona 2 improves from 0.71 to 0.89. These cases demonstrate that when user preferences are specific and well-represented in the collection, the reranking mechanism can substantially reshape the result list.

Interpretability and architecture. Our scoring function decomposes the final score into individually meaningful components: lexical relevance, genre overlap, author match, book-title match, repeat-click bonus and two semantic similarity terms. This transparency makes the system easy to debug and to explain to users. Our architecture, combining explicit preferences with click-derived feedback, provides a transition from cold start to interaction-rich personalization without requiring a hard switching mechanism.

Limitations in per-persona results. Not all personas benefit equally. Persona 1 sees no P@10 improvement in any setting, and its nDCG@10 decreases after clicks (0.52→0.49). For Personas 6 and 8, the No clicks setting achieves higher nDCG@10 than After clicks, suggesting that click signals can sometimes introduce noise that dilutes an already effective explicit profile. This is consistent with the known limitations of click data as implicit feedback [2] and indicates that the adaptive blending formula could benefit from a confidence-aware mechanism rather than relying solely on click count.

Evaluation and design limitations. Our evaluation uses 12 synthetic personas with relevance labels. With only 24 query-level observations per system, the statistical power is limited. On the design side, all reranking weights are manually chosen, and the dominant user–document semantic weight ($W_{\text{usem}} = 8.0$) risks over-personalization when the query intent diverges from the user's historical profile. The fixed candidate set ($K = 20$) also bounds recall. Learning weights from held-out data and expanding candidates through dense retrieval are directions for future work.

7 Conclusion

We presented a personalized search engine that combines BM25 lexical retrieval with a reranking stage incorporating explicit user preferences, click-based implicit feedback and dense semantic similarity. Evaluated on 12 synthetic personas, the system improves average P@10 from 0.62 to 0.73 and nDCG@10 from 0.63 to 0.76 compared to the non-personalized baseline. Notably, the explicit profile alone yields a meaningful improvement in the cold-start setting, and click feedback provides further gains for most personas.

The per-persona analysis shows that personalization is not uniformly beneficial: its effectiveness depends on how specific the user's preferences are and how well they are represented in the collection. Click signals can also introduce noise when the explicit profile is already strong.

Future work could address the current limitations by learning reranking weights from held-out relevance data, incorporating dense retrieval to expand the candidate pool beyond the BM25 top-K, and evaluating with real users to better capture the diversity of actual search behavior.

References

[1] Zhicheng Dou, Ruihua Song, and Ji-Rong Wen. A large-scale evaluation and analysis of personalized search strategies. In *Proceedings of the 16th International Conference on World Wide Web*, pages 581–590, 2007.

[2] Thorsten Joachims, Laura Granka, Bing Pan, Helene Hembrooke, and Geri Gay. Accurately interpreting clickthrough data as implicit feedback. In *Proceedings of the 28th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 154–161, 2005.

[3] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[4] Jaime Teevan, Susan T. Dumais, and Eric Horvitz. Personalizing search via automated analysis of interests and activities. In *Proceedings of the 28th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 449–456, 2005.

[5] Ghazaleh H. Torbati, Andrew Yates, and Gerhard Weikum. Personalized entity search by sparse and scrutable user profiles. In *Proceedings of the 2020 Conference on Human Information Interaction and Retrieval*, pages 427–431, 2020.

A Appendix

A.1 Example Persona Definition

Table 3 shows the structure of a single evaluation persona. All 12 personas follow the same format.

Table 3: Example persona structure (Persona 1).

Field	Example value
Favorite genres	Fantasy, mythology, historical fiction
Favorite authors	Neil Gaiman, Madeline Miller
Favorite books	American Gods, Circe
Interests text	Interested in mythology, gods, folklore, and literary retellings
Warm-up queries	gods and mythology; ancient heroes; retelling legends
Test queries	modern mythology novels; books about gods and mortals

A.2 Per-Query Results

Table 4 reports P@10 and nDCG@10 for each individual test query, providing a finer-grained view than the per-persona averages in Table 2.

Table 4: Per-query P@10 and nDCG@10 across all personas and evaluation settings.

		P@10			nDCG@10		
Persona	**Query**	**Base.**	**No cl.**	**After cl.**	**Base.**	**No cl.**	**After cl.**
Persona 1	Q1	0.8	0.8	0.7	0.567	0.515	0.494
	Q2	0.4	0.4	0.5	0.465	0.614	0.487
Persona 2	Q1	0.6	0.7	0.8	0.840	0.985	0.914
	Q2	0.7	0.8	0.9	0.573	0.621	0.861
Persona 3	Q1	0.8	0.9	0.8	0.729	0.837	0.867
	Q2	0.6	0.6	0.7	0.798	0.788	0.796
Persona 4	Q1	0.8	0.7	0.8	0.797	0.767	0.877
	Q2	0.4	0.5	0.5	0.380	0.527	0.529
Persona 5	Q1	0.2	0.4	0.4	0.524	0.482	0.501
	Q2	0.8	0.8	0.8	0.688	0.696	0.691
Persona 6	Q1	0.4	0.6	0.7	0.601	0.883	0.873
	Q2	0.8	0.8	0.9	0.906	0.991	0.959
Persona 7	Q1	0.5	0.5	0.5	0.490	0.624	0.685
	Q2	0.6	0.7	0.7	0.776	0.739	0.739
Persona 8	Q1	1.0	0.9	0.9	0.762	0.906	0.797
	Q2	0.7	0.7	0.9	0.537	0.891	0.890
Persona 9	Q1	0.7	0.7	0.7	0.609	0.651	0.654
	Q2	0.5	0.7	0.8	0.557	0.651	0.831
Persona 10	Q1	0.4	0.4	0.5	0.380	0.578	1.000
	Q2	0.7	0.8	0.8	0.655	0.810	1.000
Persona 11	Q1	0.5	0.8	0.8	0.558	0.751	0.892
	Q2	0.6	0.7	0.8	0.589	0.482	0.534
Persona 12	Q1	0.7	0.8	0.9	0.609	0.605	0.650
	Q2	0.7	0.8	0.8	0.735	0.603	0.660